7、静态类成员

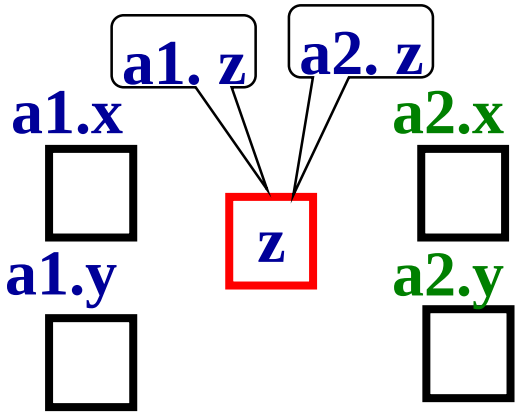
7、静态类成员

- static修饰的类成员称为静态类成员。
- 类的静态成员为所有对象共享；
- 每个对象的普通数据成员有独自内存，对于静态数据成员不管有多少对象，只有一个存在公共内存中，这个空间是在编译的时候分配的。 在说明对象时，并不为静态类型的成员分配空间；

7、静态类成员


```
class A{
    int x,y;
public:
    void Setxy(int a, int b)
    {  x=a;   y=b;}
     static  int z;
};
int A::z;
int  main(){
      A   a1,  a2;
      return 0;
}
```

不同对象，同一空间



7、静态类数据成员

- 当类中没有定义静态数据成员时，在程序执行期间遇到说明类的对象时，才为对象的所有成员依次分配存储空间；
- 当类中定义了静态数据成员时，在编译时，就要为类的静态数据成员分配存储空间。
- 必须在文件作用域中，对静态数据成员作一次且只能作一次定义性说明；
- 通过静态数据成员名前加上类名和作用域运算符，可直接引用静态数据成员。

7、静态类成员

静态数据成员和全局变量都是放在全局变量区的，但全局变量在程序中的任何位置都可以访问它，而静态数据成员受到访问权限的约束。必须是public权限时，才可能在类外进行访问。

在C++中，静态变量缺省的初值为0，静态数据成员总有唯一的初值。

为了保持静态数据成员取值的一致性，通常在构造函数中不给静态数据成员置初值，而是在对静态数据成员的定义性说明时指定初值。

7、静态类成员函数

静态成员函数

用关键字static来修饰成员函数，可以将类的成员函数定义为静态的成员函数。

```
class A
{    float x, y;
  public :
     A( ){  }
     static   void sum(void)  { ..... }
};
```

7、静态类成员函数

与静态数据成员一样，在类外的程序代码中，通过类名加上作用域操作符，可直接调用静态成员函数。

静态成员函数的实现部分在类定义之外定义时，其前面不能加修饰词static。

静态成员函数没有this指针。

静态成员函数，不能访问非静态成员。

静态成员示例1

```
class  A{
     int  i,j;
    static int z;
public:
     static int x,y;
    A(int a=0,int b=0)
   {    i=a;  j=b;
        x++;  y+=2;   z*=3; }
    void Show()     {
     cout << i<<'\t'<<j<<'\t';
    cout <<x<<'\t'<<y<<'\t';
     }
static void show()
   {     cout<<z<<endl;   }
};
int A::x;
int A::y;
int A::z=1;
```

必须对静态成员变量进行一次说明

```
int  main()
{
   cout<<A::x<<'\t'<<A::y <<endl;
    A  a(2,3);
    a.Show();
    A::show();
    A  b(100,200);
    b.Show();
    A::show();
    a.Show();
   return 0;
   }
```

```
0       0
2       3       1       2       3
100     200     2       4       9
2       3       2       4
```

静态成员示例2

```
class  A
{    int  i;
     static int count;
    public:
     A(int a=0) {
        i=a; count++;
        cout <<"Number of Objects="<<count<<'\n';        }
     ~A() { count--; cout <<"Number of Objects="<<count<<'\n';}
     void Show()
     {   cout << "i="<<i<<'\n';    cout << "count="<<count<<"\n";}
};
int A::count;
int  main()
{          A  a1(100);
           A b[2];
           a1.Show();
          return  0;
}
```

```
Number of Objects=1
Number of Objects=2
Number of Objects=3
i=100
count=3
Number of Objects=2
Number of Objects=1
Number of Objects=0
```

静态成员示例2



主函数增加两行代码：
A c(a1);
c.show();
输出什么结果？

```
Number of Objects=1
Number of Objects=2
Number of Objects=3
i=100
count=3
i=100
count=3
Number of Objects=2
Number of Objects=1
Number of Objects=0
Number of Objects=-1
请按任意键继续. . .
```

添加一个复制构造函数：

```
A(A &r)
{
    i=r.i;
    count++;
}
```

```
Number of Objects=1
Number of Objects=2
Number of Objects=3
i=100
count=3
i=100
count=4
Number of Objects=3
Number of Objects=2
Number of Objects=1
Number of Objects=0
请按任意键继续. . .
```

计数错误？
Why?

类对象作为函数参数

对象参数

函数定义：

```
void Fun1(Point p)
{   cout<<p.GetX()<<endl; }
```

调用：

```
Point P1(2,4);
Fun1(P1);
```

对象指针

函数定义：

```
void Fun1(Point *p)
{   cout<<p->GetX()<<endl; }
```

调用：

```
Point P1(2,4),;
Fun1(&P1);
```

对象引用

函数定义

```
void Fun1(Point & p)
{   cout<<p.GetX()<<endl; }
```

调用：

```
Point P1(2,4);
Fun1(P1);
```

常对象和常成员

常对象

```
Clock  const myClock;
const Clock myClock;
```

常对象不能变，只能调用常成员函数，通常在函数参数时使用常引用。

常数据成员

常数据成员只能通过初始化列表来获得初值

常成员函数

常成员函数不能修改对象数据成员的值，也不能调用没有用const修饰的成员函数

```
class Point
{
    const int X,Y;
public:
    Point(int a,int b):X(a),Y(b){ }
    void Init(int initX,int initY);
    int GetX() const;
    int GetY() const;
};
```

8、友元

8、友元

类中私有和保护的成员在类外不能被访问。

友元是C++提供的一种破坏数据封装和数据隐藏的机制。

通过将一个模块声明为另一个模块的友元，一个模块能够引用到另一个模块中被隐藏的信息。

- 友元可分为友元函数和友元类。
- 为了确保数据的完整性，及数据封装与隐藏的原则，建议尽量不使用或少使用友元。

8.1、友元函数

友元函数是一种定义在类外部的普通函数，其特点是能够访问类中私有成员和保护成员，即类的访问权限的限制对其不起作用。

友元函数需要在类体内进行说明，在前面加上关键字**friend**：

friend 返回类型 函数名（参数表）

友元函数不是成员函数，用法也与普通的函数完全一致，只不过它能访问类中所有的数据。

8.1、友元函数

一个类的友元函数可以自由地用该类中的所有成员：

- 友元函数不受类中访问权限关键字的限制，可以把它放在类的私有部分，放在类的公有部分或放在类的保护部分，其作用都是一样的。换言之，在类中对友元函数指定访问权限是不起作用的。

友元函数不带有this指针，因此必须将对象名或对象的引用作为友元函数的参数，才能访问对象的成员。

友元函数的作用域与一般函数的作用域相同。

8.1、友元函数

友元函数与一般函数的不同点在于：

1. 友元函数必须在类的定义中说明，其函数体可在类内定义，也可在类外定义；

2. 它可以访问该类中的所有成员（公有的、私有的和保护的），而一般函数只能访问类中的公有成员。

！谨慎使用友元函数

通常使用友元函数来取对象中的数据成员值，而不修改对象中的成员值，则肯定是安全的。

8.1、友元函数

```
class Point{
        float  x,y;
public:
    Point(float a, float b){  x=a; y=b;}
    float  Sum(){  return  x+y;  }
    float  Sumx( Point &b){  return  x+b.x;  }
    friend  float Sum(Point &a){    return   a.x+a.y;        }
    friend  float Sumx(Point &a, Point &b){    return   a.x+b.x;  }
};
int main(void)
{  Point  t1(4,5),t2(10,20);
    cout<<t1.Sum()<<endl; cout<<t1.Sumx(t2)<<endl;
   cout<<Sum(t2)<<endl; cout<<Sumx(t1,t2)<<endl;
    return 0; }
```

友元函数只能通过对象名引用类中的数据。

成员函数

友元函数

私有数据

成员函数的调用，利用对象名调用

友元函数的调用，直接调用

8.1、友元函数

```
class Point{
        float  x,y;
public:
  Point(float a, float b){  x=a; y=b;}
 float  Getx(){  return x;  }
 float  Gety(){  return y; }
 float  Sum(){  return  x+y;  }
 friend  float Sum(Point &);
};
float  Sumxy(Point  &a){ return a.Getx()+a.Gety();  }
float Sum(Point &a){  return a.x+a.y;  }
int main()
{  Point  t1(1,2),t2(10,20), t3(100,200);
    cout<<t1.Sum()<<endl;
   cout<<Sum(t2)<<endl;
   cout<<Sumxy(t3)<<endl;
  return 0;
}
```

成员函数

友元函数

普通函数，必须通过公有函数访问私有成员

友元函数,可以直接调用类中私有成员

对象调用成员函数

调用友元函数

调用一般函数

8.1、友元函数

友元函数可以是某个类的成员函数，即A类中的某个成员函数是B类中的友元函数，这个成员函数可以直接访问B类中的私有数据。这就实现了类与类之间的沟通。

```
class A{
...
void fun( B &);
};
```

既是类A的成员函数

```
class B{
...
friend void A::fun( B &);
};
```

又是类B的友元函数

注意：一个类的成员函数作为另一个类的友元函数时，应先定义友元函数所在的类。

友元函数示例2


```
class B;
class A{
    float  x,y;
    public:
     A(float a, float b){  x=a; y=b;}
     float Sum(){return x+y;}
     float Sum(B &);
};
class B{
     float  m,n;
     public:
      B(float a,float  b){  m=a;n=b;  }
      friend  float A::Sum(B  &);
};
float  A::Sum( B  &b) {    x=b.m+b.n;   y=b.m-b.n; return x+y }
int main()
{    A   a1(3,5);   cout<<a1.sum()<<endl;
     B   b1(10,20);
     cout<<a1.Sum(b1)<<endl;
    return 0;          }
```

先定义类A，首先对类B作引用性说明

类A中有一个函数可以直接引用类B的私有成员

说明类A的成员函数是类B的友元函数

直接引用类B的私有成员

a1.x=30

a1.y=-10

类A的成员函数，用类A的对象调用

8.2、友元类

友元类

```
class A{
    .....
    friend class B;
};
```

```
class B{
    .....
};
```

类B是类A的友元

类B可以自由使用类A中的成员

对于类B而言，类A是透明的

类B必须通过类A的对象使用类A的成员

友元类示例

类B中的任何函数都能使用类A中的所有私有成员。

```
const float PI =3.1415926;
class  A{
            float r , h;
public:   A(float a,float b){r=a; h=b;}
            float Getr(){return r;}
            float Geth(){return h;}
            friend class B;
};
class B{
    int number;
  public:
   B(int n=1) {number=n;}
    void Show(A &a){ cout<<PI*a.r*a.r*a.h*number<<endl;}
};
int  main()
{    A a1(25,40),a2(10,40);
     B b1(2);
     b1.Show (a1);          b1.Show (a2);
     return 0;
}
```

定义类B为类A的友元

直接引用类A的私有成员

8.2、友元类

- **友元关系是不能传递的**

  B 类是 A 类的友元

  C 类是B 类的友元

  但： **C**类与 A 类没有友元关系

- **友元关系是单向的**

  B 类是 A 类的友元：

  B 类的成员可访问 A 类所有成员；

  但 A 类不能直接访问**B**类的成员。

9、运算符重载

9、运算符重载

【回顾】函数的重载

函数的重载是指完成不同功能的函数可以具有相同的函数名。

C++的编译器是根据函数的实参来确定应该调用哪一个函数的。

重载：参数个数不同或者类型不同

```
int  fun(int a,  int b)
{   return a+b;  }
int  fun (int a)
{   return a*a;  }
```

```
void  main(void)
{  cout<<fun(3,5)<<endl;
   cout<<fun(5)<<endl;
}
```

输出：

8

25

9、运算符重载

C++预定义的运算符，只能用于基本数据类型的运算

C++允许通过重新定义运算符，使它能够用于特定类的对象执行特定的功能。

```
class Complex
{  double real,image;
public:
   Complex(double r, double i)
    {  real=r; image=i; }
};
int main(void)
{  Complex   c1(2.0,3.0),  c2(3.0,4.0),  c3;
    c3=c1+c2;
    return 0;
}
```

Error 两对象不能直接使用＋

运算符重载就是赋予已有的运算符多重含义。

9、运算符重载

运算符的重载实质是函数重载，定义一个函数，把运算符的操作数转换为函数的参数，由这个函数来完成该运算符应该完成的操作。这种函数称为运算符重载函数。

运算符重载函数定义格式如下：

返回值类型 operator 重载的运算符（参数表）

{ 函数体 }

运算的对象

关键字

运算符可以重载为普通函数，也可以重载为成员函数。

9、运算符重载

复数运算符重载完整示例

```
class Complex
{   double real,image;
    public:
   Complex(double r=0, double i=0)
   {  real=r; image=i; }
   void AddComplex(Complex &c1,Complex &c2)
  {  real=c1.real+c2.real;  image=c1.image+c2.image;}
   Complex operator+(Complex &c)
   { return Complex(real+c.real,image+c.image);}
};
        int main()
            Complex c1(1.0,1.0),c2(2.0,3.0),c3,c4;
            c3.AddComplex(c1,c2);
            c4=c1+c2;
            return 0;        }
```

运算符重载为成员函数时，默认左操作数是当前对象

调用专门的功能函数

重载了加法运算符，可直接进行类的运算

c4=c1.operator+(c2)

9、运算符重载

当用成员函数实现运算符的重载时，运算符重载函数的参数只有二种情况：没有参数或带有一个参数。

- 对于只有一个操作数的运算符(如++)，在重载这种运算符时，通常不能有参数；
- 对于有二个操作数的运算符，只能带有一个参数。

在C++中不允许重载有三个操作数的运算符

只能对C++中已定义了的运算符进行重载，而且当重载一个运算符时，该运算符的优先级和结合律是不能改变的

9、运算符重载


```
class Complex
{
        double Real,Image;
public:
        Complex(double r=0.0,double i=0.0)
        {   Real=r;              Image=i;       }
        Complex(Complex &com)
        {  Real=com.Real;   Image=com.Image;  }
        void Print()
        { cout<<Real<<‘+'<<Image<<‘i'<<endl;  }
        Complex operator =(const Complex &);
        Complex operator +(const Complex &);
        Complex operator -(const Complex &);
        Complex operator *(const Complex &);
        Complex operator /(const Complex &);
        double abs();
};
```

9、运算符重载

++运算符的重载

- **++为前置运算时，它的运算符重载函数的一般格式为：**

返回类型 operator ++( )

{ ……; }

- **++为后置运算时，它的运算符重载函数的一般格式为：**

返回类型 operator ++(int)

{ ……; }

9、运算符重载为友元函数

友元函数是在类外的普通函数，与一般函数的区别是可以调用类中的私有或保护数据

如果将运算符的重载函数为类的成员函数，实际上是由一个操作数调用另一个操作数。

```
A  a ,b ,  c;
c=a+b;        实际上是c=a.operator+(b);
c+=a;         实际上是c.operator+=( a );
```

若运算符的重载函数定义为友元函数，参与运算的对象全部成为函数参数。

```
c=a+b;        实际上是 c=operator+(a, b);
c+=a;         实际上是 operator+=( c, a );
```

9、运算符重载为友元函数

对双目运算符，友元函数有2个参数，对单目运算符，友元函数有一个参数。有些运算符不能重载为友元函数，它们是：＝，（），[ ]，->等

对双目运算符，重载为成员函数时，仅一个参数，另一个被隐含；重载为友元函数时，有两个参数，没有隐含参数。

运算符重载为友元函数示例1

```
class A
{       int i;
   public:
        A(int a=0)      { i=a;   }
        void Show()   {cout<<"i="<<i<<endl;       }
        friend A operator +(A &,A &);
};
A operator +(A &a , A &b)
        {A t;    t.i=a.i+b.i;      return t; }
int main()
{       A a1(10),a2(20),a3;
        a1.Show ();
        a2.Show ();
        a3=a1+a2;     //重新解释了加法，可以直接进行类的运算
        a3.Show ();
}
```

运算符重载为友元函数示例2

```
class Complex
{
        double Real,Image;
public:
        Complex(double r=0.0,double i=0.0)
        {  Real=r;       Image=i;        }
        Complex(Complex &com)
        {  Real=com.Real;Image=com.Image;}
        void Print()
        {    cout<<"Real="<<Real<<'\t';
          cout<<"Image="<<Image<<endl;
        }
        Complex operator +(const Complex &);
        Complex operator +=(Complex &);
        double abs();
};
```



运算符重载为友元函数示例2

```
class Complex
{
        double Real,Image;
public:
        Complex(double r=0.0,double i=0.0)
        {  Real=r;    Image=i;    }
        Complex(Complex &com)
        {  Real=com.Real;Image=com.Image;}
        void Print()
        {    cout<<"Real="<<Real<<'\t';
             cout<<"Image="<<Image<<endl;
        }
    friend Complex operator +(const Complex &,
                              const Complex &);
    friend Complex operator +=(Complex &,
                               const Complex &));
    friend double abs(Complex &);
};
```

友元函数重载运算符+，需要两个参数

运算符重载为友元函数示例2

```
Complex operator +(const Complex &c1, const Complex &c2)
{
   Complex    temp(c1.Real+c2.Real,c1.Image+c2.Image);
  return temp;
}
Complex operator +=(Complex &c1,const Complex &c2)
{
        c1.Real=c1.Real+c2.Real;
        c1.Image=c1.Image+c2.Image;
        return c1;
}
double abs(Complex &c)
{
        return sqrt(c.Real*c.Real+c.Image*c.Image);
}
```

补. 重载插入和提取运算符

- 问题：

用<<和>>可以对基本数据类型进行插入和提取，但是回顾作业里，分数和复数不能直接输出需要一个函数输出需要的格式。

解决方法： C++允许用户重载插入和提取运算符，以方便用户利用输入/输出流来输入输出自定义的数类型，实现对象的输入/输出。

东南大学信息科学与工程学院
王文博2026

补. 重载插入和提取运算符

```
#include <iostream>
using namespace std;
class Complex
{
public:
    Complex( ){real=0;imag=0;}
    Complex(double r,double i)
      {  real=r;imag=i;  }
    void display( );
 private:
    double real;
    double imag;
};
```

```
void Complex::display( )
{
   cout<<"("<<real<<","<<imag<<"i)"<<endl;
}
```


东南大学信息科学与工程学院
王文博2026

补. 重载插入和提取运算符

```
int main( )
{
    Complex c1(3,4),c2(5,-10),c3;
    c3=c1.complex_add(c2);
    cout<<"c1=";
    c1.display( );
    cout<<"c2=";
    c2.display( );
    cout<<"c1+c2=";
    c3.display( );
    return 0;
}
```

调用方式不直观，程序繁琐。

王文博2026

补. 重载插入和提取运算符

对“<<”和“>>”重载的函数形式如下：

```
friend istream & operator >> (istream &, 自定义类 &);
friend ostream & operator << (ostream &, 自定义类 &);
```

友元函数　返回值类型　函数名　左操作数　右操作数

- 重载“>>”的函数的第一个参数和函数的类型都必须是istream&类型。 cin>>a; ⟺ operator>>(cin, a)
- 重载“<<”的函数的第一个参数和函数的类型都必须是ostream&类型。 cout<<a; ⟺ operator<<(cout, a)
- >>和<<重载函数的第二个参数是要进行输出操作的类。
- 一般将重载“>>”和“<<”的函数作为友元或普通函数。

补. 重载插入和提取运算符

例：用重载的“>>”和“<<”输入输出复数

```
#include <iostream>
using namespace std;
class Complex {
  public:
    Complex( ){   real=0;   imag=0;  }
    Complex(double r,double i)
    {  real=r;   imag=i;  }
    Complex operator + (Complex &c2);
    friend ostream& operator << (ostream&, const Complex&);
    friend istream& operator >>  (istream&, Complex&);
private:
   double real;
   double imag;
};
```


东南大学信息科学与工程学院
王文博2026

补. 重载插入和提取运算符

```
Complex Complex::operator + (const Complex &c2)
{return Complex(real+c2.real,imag+c2.imag); }

ostream& operator << (ostream& output, const  Complex& c)
{
    output<<“(“<<c.real<<“+”<<c.imag<<“i)” <<endl;
    return output;
}

istream& operator >> (istream& input, Complex& c)
{
     cout<<“input real and imaginary  :”;
     input>>c.real>>c.imag;
     return input;
}
```


东南大学信息科学与工程学院
王文博2026

补. 重载插入和提取运算符

```
int main( )
{
    Complex c1,c2;
    cin>>c1>>c2;
    cout<<“c1=“<<c1<<endl;
    cout<<“c2=“<<c2<<endl;
    return 0;
}
```


东南大学信息科学与工程学院
王文博2026